ETHNUS™

Explore | Expand | Enrich

<Codemithra />™

WINNER TREE

WINNER TREE

EXPLANATION

A tournament tree comes from the idea of a tournament or a competition

If there are p players in a tournament, then the tournament tree

- Will have p leaf nodes which can also be called external nodes
- Will have p-1 internal nodes

The tournament trees are also known as selection trees

The tournament tree is an application of the binary heap data structure

WINNER TREE

EXPLANATION

The tournament tree is an application of the binary heap data structure

Hence it is a complete binary tree

Two Types of Tournament Trees

1. Winner Trees
2. Looser Trees

WINNER TREE

EXPLANATION

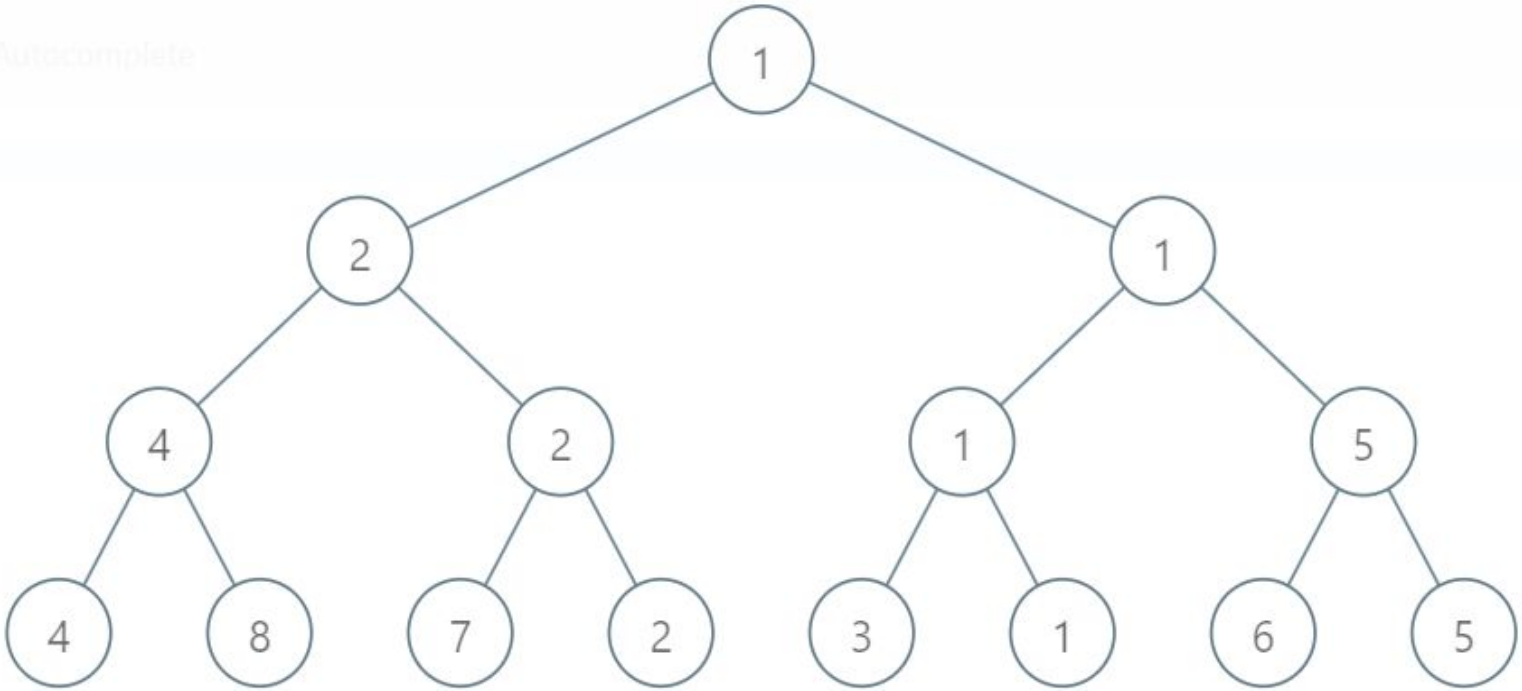
The idea here is that in a winner tree, the nodes at a particular level store the winner of the two nodes at the level below it.

Hence, the tree’s root node stores the overall winner of the competition.

For instance, if we consider that in a tournament there are a total of eight players (1, 2, 3, 4, 5, 6, 7, 8) and in a match between any two players among these, the player with numerically smaller value wins the match (minimum winner tree).

WINNER TREE

EXPLANATION



EXPLANATION-LOSER TREE

- The loser tree is another type of tournament tree in which the nodes at a particular level store the loser of the two nodes at the level below it.

- In a Loser tree, the tournament’s winner is stored at the top, and the second place (runner-up) is the child of it.

- So, if we are considering a numerically smaller value as the winner, then each node’s value in a Loser Tree will be the greater value among both of its children.

EXPLANATION-LOSER TREE

One advantage of loser trees compared to winner trees is that it is sufficient to examine the nodes on the path from leaf nodes to the root node for restructuring the tree in looser trees.

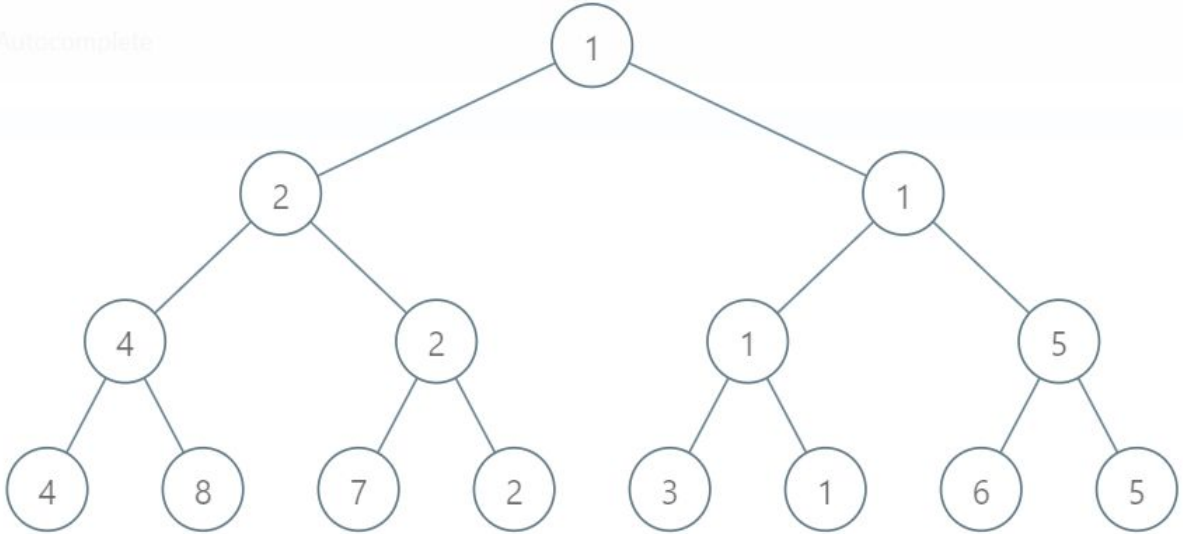
To form a looser tree, we have to create winner tree at first. We will store looser of the match in each internal node.

Loser Trees can be used to find the runner up of a tournament

For each of the matches, the time complexity is O(1), and in total, there are n-1 nodes (internal nodes); hence the time complexity to initialize the tree is O(n).

WINNER TREE

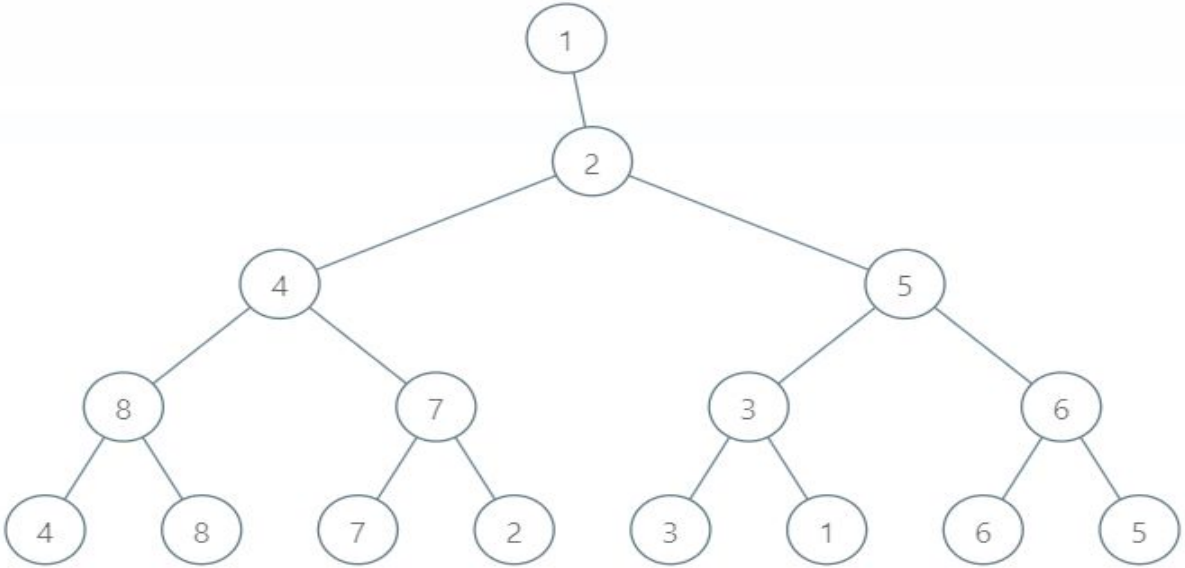
EXPLANATION



For this winner tree

WINNER TREE

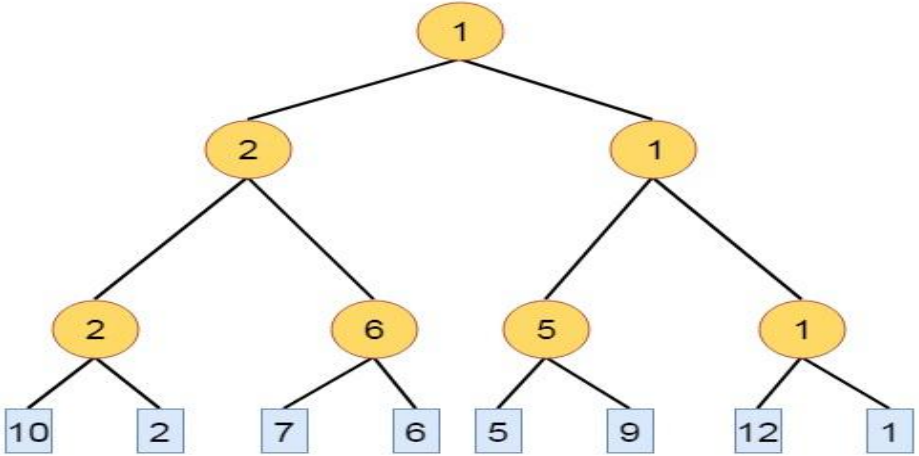
EXPLANATION



For this winner tree

WINNER TREE

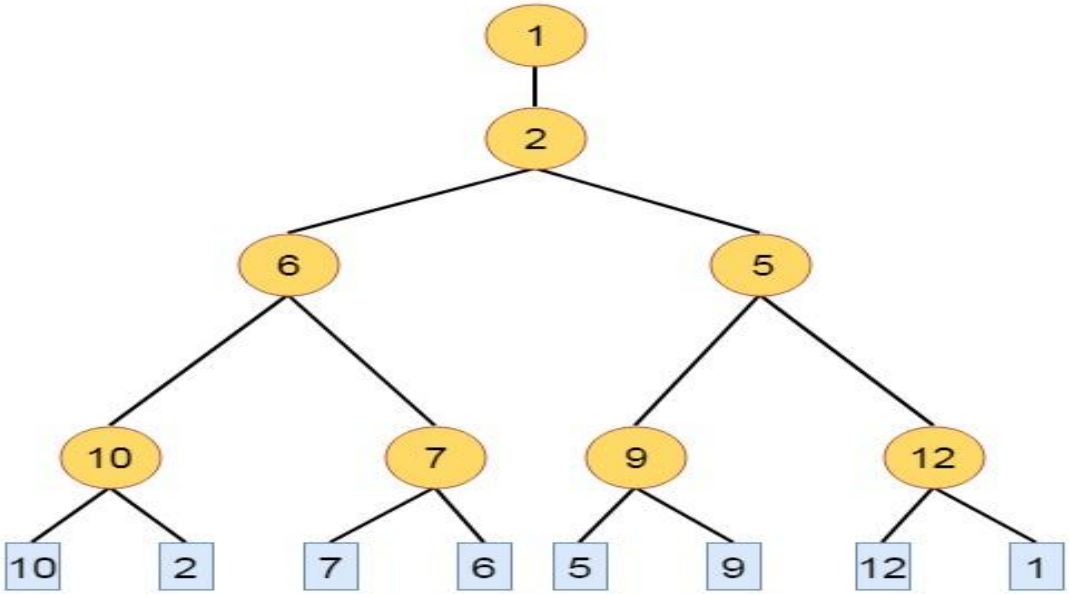
EXPLANATION



For this winner tree for the elements 10, 2, 7, 6, 5, 9, 12, 1.

WINNER TREE

EXPLANATION



For this Loser tree

WINNER TREE

Suppose you are given an array representing the scores of players in a competition. Each player is represented by an index in the array, and their corresponding score is the value at that index. Write a Java program to implement a Winner Tree data structure that efficiently finds the index of the player with the highest score. Provide a detailed explanation of how the Winner Tree is constructed and how the winner is determined. Test your program with the following array of player scores: {3, 7, 1, 9, 4, 2, 8, 5}.

WINNER TREE-A1

```
public class WinnerTree {
    private int[] tree;
    private int[] players;
    private int numPlayers;

    public WinnerTree(int[] players) {
        this.players = players;
        this.numPlayers = players.length;
        int treeSize = 2 * numPlayers - 1;
        this.tree = new int[treeSize];
        buildWinnerTree();
    }

    private void buildWinnerTree() {
        for (int i = numPlayers - 1; i <
tree.length; i++) {
            tree[i] = i - (numPlayers - 1);
        }
```

```
for (int i = numPlayers - 2; i >= 0; i--) {
            tree[i] = players[tree[2 * i + 1]]
< players[tree[2 * i + 2]] ? tree[2 * i + 2] :
tree[2 * i + 1];
        }
    }
public int getWinner() {
        return players[tree[0]];
    }

    public static void main(String[] args) {
        int[] players = {3,7,1,9,4,2,8,5};
        WinnerTree winnerTree = new
WinnerTree(players);
        System.out.println("The winner is: " +
winnerTree.getWinner());
    }
}
```

WINNER TREE-A2

```java
import java.util.Arrays;
public class Main {
    public static class WinnerTree {
        private int[] tree;
        private int[] players;

        public WinnerTree(int[] players) {
            this.players = players;
            int n = players.length;
            int treeSize = 1;
            while (treeSize < n) {
                treeSize *= 2;
            }
            tree = new int[2 * treeSize - 1];
            Arrays.fill(tree, -1);
            for (int i = 0; i < n; i++) {
                tree[treeSize - 1 + i] = i;
            }
            build(0, 0, treeSize - 1);
        }
```

```java
private void build(int node, int left, int right) {
            if (left == right) return;
            int mid = (left + right) / 2;
            build(2 * node + 1, left, mid);
            build(2 * node + 2, mid + 1, right);
            tree[node] = players[tree[2 * node + 1]] <= players[tree[2 * node + 2]] ? tree[2 * node + 1] : tree[2 * node + 2];
        }
public int getWinner() {
            return tree[0];
        }
    }
```

WINNER TREE-A2

```
public static void main(String[] args)
{
        int[] players = {3, 7, 1, 9,
4, 2, 8, 5};         WinnerTree
winnerTree = new WinnerTree(players);
        int winnerIndex =
winnerTree.getWinner();
        System.out.println("The player
with the highest score is at index: "
+ winnerIndex);
        System.out.println("The score
of the winning player is: " +
players[winnerIndex]);
    }
}
```

INTERVIEW QUESTIONS

1. What is a winner tree, and how does it work?

Answer: A winner tree is a binary tree where each internal node represents the minimum value among its children. It efficiently identifies the overall winner (minimum) among a set of elements. The root of the tree holds the ultimate winner.

INTERVIEW QUESTIONS

2.How is a winner tree different from a loser tree?

Answer: Winner trees find the minimum value, while loser trees find the maximum. Winner tree nodes represent the winner of their children, while loser tree nodes represent the loser.

INTERVIEW QUESTIONS

3. Explain the process of updating a value in a winner tree.

Answer: To update a value in a winner tree:

- Modify the corresponding leaf node.
- Propagate the change to the root, recalculating winners at each level.

INTERVIEW QUESTIONS

4.Provide an example use case where a winner tree would be beneficial.

Answer: Winner trees are useful in ranking scenarios, like sports tournaments, where you need to quickly identify the best performer with the lowest score or fastest time.

INTERVIEW QUESTIONS

5.Are winner trees limited to numeric values, or can they handle other types of data?

Answer: Winner trees are versatile and can handle various data types, not limited to numeric values. The comparison criteria used to determine winners can be adapted for different data types.

https://learn.codemithra.com

ETHNUS™
Explore | Expand | Enrich

THANK YOU

+91 78150 95095 | codemithra@ethnus.com | www.codemithra.com